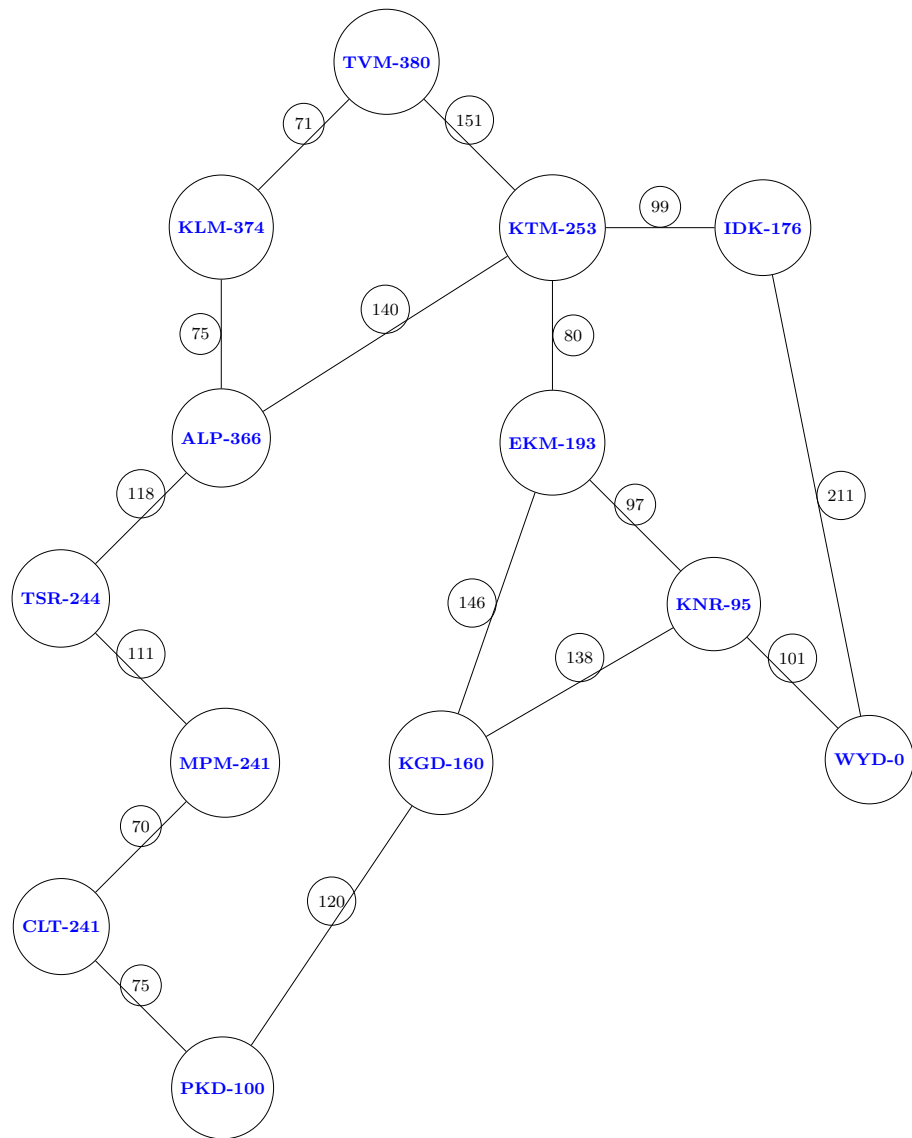


A* Algorithm

(Heuristic Search for Finding Shortest Path)

April 16, 2025



3.3.2 The A* Algorithm

The best-first search algorithm that was just presented is a simplification of an algorithm called A*, which was first presented by Hart *et al.* (1968; 1972). This algorithm uses the same f' , g , and h' functions, as well as the lists **OPEN** and **CLOSED**, that we have already described.

Algorithm: A*

1. Start with **OPEN** containing only the initial node. Set that node's g value to 0, its h' value to whatever it is, and its f' value to $h' + 0$, or h' . Set **CLOSED** to the empty list.
2. Until a goal node is found, repeat the following procedure: If there are no nodes on **OPEN**, report failure. Otherwise, pick the node on **OPEN** with the lowest f' value. Call it **BESTNODE**. Remove it from **OPEN**. Place it on **CLOSED**. See if **BESTNODE** is a goal node. If so, exit and report a solution (either **BESTNODE** if all we want is the node or the path that has been created between the initial state and **BESTNODE** if we are interested in the path). Otherwise, generate the successors of **BESTNODE**, but do not set **BESTNODE** to point to them yet.
3. For each such **SUCCESSOR**, do the following:
 - (a) Set **SUCCESSOR** to point back to **BESTNODE**. These backwards links will make it possible to recover the path once a solution is found.
 - (b) Compute $g(\text{SUCCESSOR}) = g(\text{BESTNODE}) + \text{cost of getting from BESTNODE to SUCCESSOR}$.
 - (c) See if **SUCCESSOR** is the same as any node on **OPEN** (i.e., it has already been generated but not processed). If so, call that node **OLD**. Since this node already exists in the graph, we can throw **SUCCESSOR** away and add **OLD** to the list of **BESTNODE**'s successors. Now we must decide whether **OLD**'s parent link should be reset to point to **BESTNODE**. It should be if the path we have just found to **SUCCESSOR** is cheaper than the current best path to **OLD** (since **SUCCESSOR** and **OLD** are really the same node). See whether it is cheaper to get to **OLD** via its current parent or to **SUCCESSOR** via **BESTNODE** by comparing their g values. If **OLD** is cheaper (or just as cheap), then we need do nothing. If **SUCCESSOR** is cheaper, then reset **OLD**'s parent link to point to **BESTNODE**, record the new cheaper path in $g(\text{OLD})$, and update $f'(\text{OLD})$.
 - (d) If **SUCCESSOR** was not on **OPEN**, see if it is on **CLOSED**. If so, change its parent to **BESTNODE** and add **OLD** to the list of **BESTNODE**'s successors. Check to see if the new path or the old

path is better, just as in step 2(c), and set the parent link and g and f' values appropriately. If we have just found a better path to **OLD**, we must propagate the improvement to **OLD**'s successors. This is a bit tricky. **OLD** points to its successors. Each successor in turn points to its successors, and so forth, until each branch terminates with a node that either is still on **OPEN** or has no successors. To propagate the new cost downward, do a depth-first traversal of the tree starting at **OLD**, changing each node's g value (and thus also its f' value), terminating each branch when you reach either a node with no successors or a node to which an equivalent or better path has already been found. This condition is easy to check for. Each node's parent link points back to its best known parent. As we propagate down to a node, see if its parent points to the node we are coming from. If so, continue the propagation. If not, then its g value already reflects the better path of which it is part. So the propagation may stop here. But it is possible that with the new value of g being propagated downward, the path we are following may become better than the path through the current parent. Compare the two. If the path through the current parent is still better, stop the propagation. If the path we are propagating through is now better, reset the parent and continue propagation.

- (e) If **SUCCESSOR** was not already on either **OPEN** or **CLOSED**, then compute $f'(\text{SUCCESSOR}) = g(\text{SUCCESSOR}) + h'(\text{SUCCESSOR})$.